

Deep Neural Networks

Lab 2 – Building a Neural Network with Keras

Module: CMP6232 Deep Neural Networks and Ethics

Session length: 3 hours

Introduction

In this lab you will use Keras to build a neural network that recognises the MNIST handwritten digits. You will start with a simple single-layer model, then improve it by adding hidden layers and experimenting with the settings.

In this lab you will

- Load and preprocess the MNIST dataset.
- Build, compile, train and evaluate a first neural network in Keras.
- Improve the model by adding hidden layers.
- Experiment with the depth and width of the network, and visualise how it trains.

Dataset Overview

The MNIST dataset is included in Keras and can be loaded directly. It contains handwritten digits (0–9) and is split into two parts:

- **Training set** – used to fit (train) the neural network.
- **Test set** – used to evaluate the model on unseen data.

The data must be preprocessed before it is fed to the network. Pixel values are converted to float32 (to support GPU computation) and normalised to the range [0, 1]. The dataset also includes the true labels (the ground truth), which we one-hot encode.

Building the Model

Step 1: Import the required libraries

First, import the modules needed to build the model:

```
from __future__ import print_function
import tensorflow as tf
import numpy as np
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Activation
from tensorflow.keras.optimizers import SGD
from tensorflow.keras.utils import to_categorical

np.random.seed(1671) # for reproducibility
```

Step 2: Define the training parameters

Set up the parameters that control training:

```
NB_EPOCH = 20          # number of training epochs
BATCH_SIZE = 128        # batch size for training
VERBOSE = 1             # verbosity mode
```

```
NB_CLASSES = 10      # number of classes (one per digit, 0-9)
OPTIMIZER = SGD()    # SGD optimiser
VALIDATION_SPLIT = 0.2 # fraction of TRAIN reserved for validation
```

Step 3: Load and preprocess the data

Load the data, then reshape each 28×28 image into a flat vector of 784 values and normalise the pixels to [0, 1].

```
# data: shuffled and split between train and test sets
(X_train, y_train), (X_test, y_test) = mnist.load_data()

RESHAPED = 784
X_train = X_train.reshape(60000, RESHAPED)
X_test = X_test.reshape(10000, RESHAPED)
X_train = X_train.astype('float32')
X_test = X_test.astype('float32')

# normalise pixel values to the [0, 1] range
X_train /= 255
X_test /= 255

print(X_train.shape[0], 'train samples')
print(X_test.shape[0], 'test samples')
```

Why normalise Each pixel intensity (0–255) is divided by 255, so all inputs sit between 0 and 1. Networks train faster and more reliably when inputs are on the same small scale.

Step 4: Convert the labels to one-hot encoding

Convert the class labels into binary class matrices (one-hot vectors):

```
Y_train = to_categorical(y_train, NB_CLASSES)
Y_test = to_categorical(y_test, NB_CLASSES)
```

Step 5: Build the neural network model

Build a first, simple model: a single dense layer with a softmax output.

```
model = Sequential()
model.add(Dense(NB_CLASSES, input_shape=(RESHAPED,)))
model.add(Activation('softmax'))
model.summary()
```

Understanding the model:

- The input has 784 values, one per pixel.
- The output layer has 10 neurons, one per digit.
- Softmax turns the outputs into class probabilities.

Step 6: Compile the model

Before training, specify the loss function, the optimiser and the metrics to track:

```
model.compile(loss='categorical_crossentropy', optimizer=OPTIMIZER,
              metrics=['accuracy'])
```

- **Loss function** – measures the error (categorical cross-entropy for multi-class).
- **Optimiser** – decides how the weights are updated during training.
- **Metrics** – accuracy is used to track performance.

Step 7: Train the model

Use `fit()` to train the model. The main hyperparameters are the number of epochs and the batch size:

```
history = model.fit(X_train, Y_train, batch_size=BATCH_SIZE,
                    epochs=NB_EPOCH, verbose=VERBOSE,
                    validation_split=VALIDATION_SPLIT)
```

- **epochs** – how many times the model sees the whole training set.
- **batch_size** – how many samples are seen before each weight update.

Step 8: Evaluate the model

Evaluate the trained model on the test set. The test set should contain only unseen examples:

```
score = model.evaluate(X_test, Y_test, verbose=VERBOSE)
print("Test score:", score[0])
print('Test accuracy:', score[1])
```

How accurate is your model? Note the accuracy. What percentage of the handwritten digits are classified correctly? Keep this number to compare against your improved models.

Improving the Model

First attempt: adding hidden layer(s)

You can improve the model by adding layers. After the input, add a dense hidden layer with `N_HIDDEN` neurons, then a second hidden layer, then the output layer of 10 neurons. A layer is called hidden because it connects to neither the input nor the output directly.

```
N_HIDDEN = 128
model = Sequential()
model.add(Dense(N_HIDDEN, input_shape=(RESHAPED,)))
model.add(Dense(N_HIDDEN))
model.add(Dense(NB_CLASSES))
model.add(Activation('softmax'))
model.summary()
```

This is now a deep neural network. You can keep adding hidden layers to make it deeper. Recompile and retrain it (Steps 6 and 7) to see how the accuracy changes.

Experiment with the architecture

Stay with this model for now and change only its shape (depth and width):

1. **Add more hidden layers** – does a deeper network improve accuracy?
2. **Change the neurons per layer** – try `N_HIDDEN = 256` or `512`. Does a wider network help?

You will probably find accuracy improves only a little. The reason, and the fixes, come in Week 3.

Extension Activities

Activity 1: Plot the learning curves

The `fit()` call returns a history object that records the accuracy and loss at each epoch. Plotting it shows whether the model is still improving or starting to overfit. Run this after Step 7.

```
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='train')
plt.plot(history.history['val_accuracy'], label='validation')
plt.title('Accuracy'); plt.xlabel('epoch'); plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='train')
plt.plot(history.history['val_loss'], label='validation')
plt.title('Loss'); plt.xlabel('epoch'); plt.legend()
plt.show()
```

Try this

- Do the training and validation curves stay close, or do they separate?
- A growing gap between them is a sign of overfitting. At which epoch does it start?

Activity 2: Count the parameters

`model.summary()` prints a table of the layers and a Param # column: the number of weights the network has to learn. This tells you how big your model really is. Run it for each model you build.

```
model.summary()
```

Try this

- How many parameters does the single-layer model (Step 5) have, compared with the deeper model?
- Set `N_HIDDEN` to 256, then 512, rebuild, and see how the parameter count grows.
- Most of the parameters sit in the first hidden layer. Why? (Hint: there are 784 inputs.)

Before You Leave

Looking ahead to Week 3 Even with extra layers, this network is still linear, so the accuracy stalls. Next week you will add non-linearity (ReLU), dropout and better optimisers to push it much further.

- You have trained and evaluated the simple model, and recorded its accuracy.
- You have built and retrained a deeper model with hidden layers.
- You have changed the depth or width of the network and noted the effect.
- You have saved your notebook to Google Drive (File → Save a copy in Drive).